

Software Engineering

Take-Home Assignment

LLM-Powered Workflow Builder

Time Estimate

5–7 hours of focused work. We respect your time—scope accordingly.

Deadline: 3 calendar days from receipt of this document.

Overview

Build a **conversational workflow builder** where a user constructs a deterministic, multi-step automation entirely through a chat interface powered by a large language model (LLM). The user should never leave the chat to configure actions, connect apps, or define logic. The final output of every conversation is a **fully-resolved JSON workflow definition** that could be executed by a headless runner without further human input.

What We Mean by “Workflow”

A workflow is a directed sequence of steps that runs deterministically once triggered. Each step represents a discrete action—calling an API, transforming data, routing based on a condition—and its configuration is fully resolved at build time, not at execution time. Think of it as a recipe: the user writes the recipe in conversation with the LLM, and the output JSON is the recipe card that any cook (runner) can follow without improvisation.

A valid workflow definition must include: a **trigger** (what starts the workflow), one or more **action steps** (what happens), any **conditional branches** or transforms in between, and all necessary **authentication and configuration** resolved via connected apps.

Core Requirements

1. Chat-Driven Workflow Construction

The entire workflow authoring experience must happen inside the chat. The LLM interprets the user’s intent, asks clarifying questions, and progressively builds the workflow definition as the conversation evolves. There is no separate visual canvas, drag-and-drop editor, or form-based wizard—the chat is the interface.

- The LLM should guide the user through defining a trigger, action steps, and any branching logic.
- At each stage, the LLM should surface contextual in-chat widgets (not separate pages) to collect structured input—for example, dropdowns for selecting an app, inline forms for configuring fields, or confirmation cards for reviewing a step before committing it.
- The user should be able to go back, revise earlier steps, reorder actions, or delete steps through natural conversation.

2. Deterministic JSON Output

The final artifact of every conversation is a JSON document that fully describes the workflow. This JSON must be deterministic—meaning it contains no ambiguity, no unresolved placeholders, and no references that require further LLM interpretation at runtime. A separate execution engine should be able to read this JSON and run the workflow end to end.

You have full freedom to design the JSON schema. We will evaluate it for clarity, extensibility, and whether it truly captures everything needed to execute the workflow without human intervention.

3. App Integration via Pipedream Connect

Use **Pipedream Connect** (<https://pipedream.com/docs/connect/>) as the integration layer for connecting third-party apps. When a user says something like “send a Slack message” or “create a row in Google Sheets,” the system should:

1. **Surface available apps and actions** from Pipedream’s component registry.
2. **Walk the user through authentication** using Pipedream Connect’s OAuth or key-based auth flows, rendered inline in the chat (e.g., a “Connect Slack” button that opens the OAuth popup and returns tokens).
3. **Collect action-specific configuration** (channel, message template, spreadsheet ID, etc.) through the chat, using widgets where appropriate.
4. **Store the resolved auth reference and configuration** in the JSON definition so execution does not require re-authentication.

4. In-Chat Widgets

Plain text chat alone is not sufficient for a good UX. The interface should render structured widgets inline when the LLM determines that structured input is more appropriate than free-form text. Examples include:

- An app selector card showing available Pipedream integrations with icons and search.
- An auth connection widget (connect button → OAuth flow → success confirmation).
- Field configuration forms for a specific action (e.g., “Which Slack channel?” with a dropdown populated from the user’s connected workspace).
- A workflow summary card that visualizes the current state of the workflow definition for review and confirmation.
- Conditional logic builders (e.g., “If status equals X, do Y; otherwise do Z”) rendered as a simple form or decision-tree widget.

You decide which widgets to build and when to surface them. We care about the intent—using the right modality for the right input—more than visual polish.

Scope and Boundaries

In Scope

- A working chat interface (web-based, can be a single-page app).
- LLM integration for interpreting user intent and driving the conversation (use any model or provider).
- Pipedream Connect for app authentication and action configuration.
- Inline chat widgets for structured data collection.
- JSON workflow definition output that is fully resolved and deterministic.
- Basic error handling: the LLM should gracefully handle ambiguous input, missing auth, or invalid configurations.

Out of Scope (Do Not Build)

- Workflow execution engine—we only care about the definition, not running it.
- Persistent storage or user accounts—in-memory or session-scoped state is fine.
- Production-grade deployment—localhost is fine; Docker is a nice-to-have.
- Extensive visual design—functional and clear beats pretty.

Example Scenario

To make the assignment concrete, here is a sample conversation flow. Your implementation does not need to match this exactly, but the end result should support interactions of this complexity:

User	I want to create a workflow that posts a summary to Slack every time a new row is added to a Google Sheet.
LLM	Great, let's build that step by step. First, we need to set up the trigger. I see you want to watch a Google Sheet for new rows. Let me pull up Google Sheets for you. [Widget: Google Sheets app connector with "Connect" button]
User	[Clicks Connect, completes OAuth]
LLM	Google Sheets is connected. Now, which spreadsheet should I watch? [Widget: Spreadsheet selector dropdown populated from user's Drive]
User	[Selects "Q3 Sales Pipeline"]

LLM	Got it—watching “Q3 Sales Pipeline” for new rows. Now let’s set up the Slack action. Do you have Slack connected? [Widget: Slack app connector]
User	[Connects Slack] Post to #sales-updates.
LLM	What should the message look like? I can pull fields from the new row—Company Name, Deal Size, Stage. [Widget: Message template builder with available fields]
User	Use this format: “New lead: {Company Name} — \${Deal Size} — Stage: {Stage}”
LLM	Here’s your complete workflow: [Widget: Workflow summary card] Trigger: New row in “Q3 Sales Pipeline” Action: Post to Slack #sales-updates Shall I finalize? You can still add steps, conditions, or revise anything.
User	Looks good—finalize it.
LLM	Done! Here is your workflow definition: [Renders final JSON]

Technical Guidance

These are suggestions, not requirements. Use whatever stack you are most productive with.

- **Frontend:** React, Vue, Svelte, or vanilla JS. A single-page app that renders chat messages and inline widgets.
- **LLM:** Any model (OpenAI, Anthropic, open-source). We care about how you structure prompts and manage conversation state, not which model you choose.
- **Pipedream Connect:** Use their SDK or API. Their docs cover OAuth token management, component discovery, and action invocation. You will need a free Pipedream account.
- **Widgets:** Custom components rendered inline in the chat, a headless UI library, or even well-structured HTML. The bar is functional clarity, not visual fidelity.

Deliverables

1. **Source code** in a Git repository (GitHub, GitLab, or a zip). Clean commit history is appreciated.

2. **README** with: setup instructions (dependencies, env vars, how to run), a brief architecture overview explaining key design decisions, and any known limitations or shortcuts taken due to time constraints.
3. **A short demo video** (3–5 minutes, screen recording with voiceover) walking through a single workflow creation end to end. Show the chat interaction, widget behavior, Pipedream connection, and the final JSON output.
4. **The final JSON output** from your demo scenario, included as a file in the repo.

Evaluation Criteria

We weight these areas roughly as follows:

Criterion	Weight	What We Look For
Conversation Design	25%	Does the LLM guide the user naturally? Is the workflow built progressively? Can the user revise earlier steps?
JSON Schema Design	20%	Is the output schema clear, complete, and extensible? Could a runner execute it without ambiguity?
Widget UX	20%	Are widgets surfaced at the right moments? Do they collect structured input effectively?
Pipedream Integration	15%	Is auth handled smoothly? Are actions configured correctly? Are resolved credentials stored in the JSON?
Code Quality	10%	Is the code well-structured, readable, and maintainable? Are there clear abstractions?
Technical Decisions	10%	Are trade-offs documented? Are prompt engineering choices thoughtful? Is the architecture pragmatic?

A Note on Polish

We are not looking for a production-ready product. We are looking for evidence that you can think through a complex problem, make good architectural choices under time pressure, and build something that works. Rough edges are fine—just call them out in your README.

Bonus (Optional)

These are not required and will not count against you if omitted. But if you have time and energy:

- Support for conditional branching (if/else) in the workflow definition, configured through conversation.
 - A “workflow preview” widget that renders the current workflow as a simple visual diagram updating in real time as the user builds it.
 - Support for data mapping between steps (e.g., passing the output of Step 1 as input to Step 2).
 - Error recovery patterns in the JSON schema (retry logic, fallback steps).
 - A “dry run” mode that validates the workflow definition against Pipedream’s API without actually executing it.
-

Questions?

If anything in this assignment is ambiguous, that is partly intentional—how you handle ambiguity is part of what we evaluate. That said, if you have a genuinely blocking question, please reach out to your hiring contact. We would rather you ask than guess and build the wrong thing.

We are excited to see what you build. Good luck!